整体架构：从拆分计划走到完成反馈

沿用当前 Django + SQLite，新增步骤、完成登记与耗时账本；一个后端应用内分模块组织。

已有：账号与任务入口

登录 / 注册、任务清单与详情

任务编辑、状态切换、状态历史

新增：分析与执行界面

完成标准 → 规则分析 → 步骤草稿

确认采纳 → 执行 → 记时 → 完成

后续：跨任务规划

个人周预算、整体负载与维护时间

自动排期、计时器、历史估算校准

统一 HTTP API 与当前用户上下文

沿用 Cookie 会话、CSRF、归属检查与版本冲突处理；新增写入沿用幂等请求键

任务与步骤

父任务 / 一层子步骤

依赖关系与状态约束

当前清单只展示父任务

规则与拆分

目标、条件与估时区间

手工草稿 / AI 草稿

用户采纳才创建步骤

执行与完成

逐次登记 / 完成时补记

完成说明、成果链接

完成撤销与操作历史

统计与历史

按有效耗时记录汇总

预计 / 实际分开展示

记录完整性由用户确认

分析子流程

同一个 SQLite 数据库 · 业务写入使用事务

现有 Task / TaskEvent + 新增 TaskStep、TimeEntry、CompletionEvent

AnalysisRun / PlanDraft / WriteReceipt；不为每个模块拆独立数据库

2B：后台分析任务 + 模型适配器

持久化任务状态、前端轮询结果

模型只生成草稿，不能写完成与耗时

新增主线：完成标准 → 步骤执行 → 实际耗时 → 用户确认完成 → 保留历史；AI 不替用户确认结果。

前端架构：完成按钮连接两份记录

完成状态回答“做完了吗”；耗时账本回答“实际投入多久”。两者可以一起提交，但含义各自独立。

任务主页 → 父任务详情

保留原有筛选与入口；详情新增“分析与拆分 / 步骤 / 耗时记录 / 完成历史”

分析与拆分面板

目标、完成标准、基础与约束
预计总耗时区间、每周计划投入
编辑草稿、选择步骤、确认采纳
新分析不会替换已有执行记录

步骤与完成面板

步骤状态、依赖、完成标准
标记完成：说明 / 成果链接可选
显示已记耗时，补记遗漏部分
全部步骤完成后提示确认父任务

实际耗时账本

选择任务本身或某个子步骤
填写投入日期、时长、可选备注
修改 / 删除自己的记录
展示总量及“完整 / 待确认”

分析草稿状态

输入 / 生成中 / 可编辑 / 已采纳
过期结果提示重新分析
生成失败仍可手工拆分

完成确认弹层

已记 90 分钟 → 只补漏记时长
暂不填写：保持“未记录 / 待确认”
误点撤销：保留已有耗时账本

数据与提交状态

已保存数据与表单草稿分离
失败保留输入；成功刷新汇总
切换账号时隔离数据与草稿

统一 API 客户端

携带会话、CSRF、对象版本和请求键；处理 401 / 404 / 409；服务器确认成功后再显示已保存

界面约定：有记录但尚未核对时显示“已记录 3 小时 · 待确认”，不能直接显示为最终总耗时。

后端架构：业务规则集中，统计只读汇总

以下接口为拟定设计；现有账号和任务接口保留。所有新增资源都沿所属父任务验证当前用户。

API 校验层：登录、归属、输入、版本、重复请求

客户端传来的任务 / 步骤编号不是权限；步骤必须属于同一父任务；时长为正整数分钟

分析与草稿服务

POST /tasks/:id/analyses

POST /drafts/:id/apply

规则分析、草稿结构校验

单次采纳事务与版本校验

步骤与完成服务

PATCH /steps/:id

POST /tasks/:id/completions

POST /steps/:id/completions

专用撤销操作记录原因

耗时账本服务

GET/POST /tasks/:id/time-entries

PATCH/DELETE /time-entries/:id

补记、改正、作废留痕

检查归属与步骤关联

统计查询服务

GET /tasks/:id/summary

有效记录求和，无记录返回空

预计 / 已记 / 完整性分别返回

不把父子汇总再次相加

写入事务边界

版本校验 → 写步骤 / 完成事件 / 耗时及修订 → 更新版本 → 写请求回执 → 提交

一次“完成 + 补记”全部成功或全部回滚；相同请求重试返回原回执

耗时变更使相应完成耗时结论待重新确认；新分析引用最新父任务版本

统计口径

按投入日期查看实际花费

周计划投入独立存放

AI 无权写入实际耗时

读取有效账本

SQLite 数据访问层

唯一请求键防重复；写入后推进父任务版本；修改记录校验记录版本；归属限定覆盖查询和修改

接口路径均带 /api 前缀；第二部分统一任务完成入口，原状态修改接口不能绕过完成登记规则。

数据架构：账本是耗时事实，完成是独立事件

父任务的实际总耗时从记录计算；不同时保存一份可手改的父任务总数，避免统计漂移。

Task · 父任务（现有表扩展）

owner / status / version（已有）

weekly_minutes：每周计划投入（已有）

新增完成标准、条件、目标日期

实际耗时通过账本聚合返回

1:N

TaskStep · 子步骤

task_id / 顺序 / 状态 / version

完成标准、预计总耗时上下限

StepDependency 保存同任务前置关系

只允许一层，不另设每周投入

关联

TimeEntry · 实际投入记录

task_id + step_id（可空）

投入日期 / duration_minutes / 备注

创建及修改时间 / version / 作废时间

一笔只归属任务本身或一个步骤

CompletionEvent · 完成登记

task_id + step_id（可空）

完成 / 撤销 / 核对、时间、前后状态

操作者、完成说明 / 成果链接（可选）

完整性确认及当时账本版本

ExecutionEvent · 操作历史

步骤状态变化、耗时增改作废

对象编号、前后值、时间、操作者

历史保留；不参与实际耗时求和

父任务状态仍记入原 TaskEvent

WriteReceipt · 幂等回执

owner / 操作类型 / request_key

请求指纹、结果对象与响应快照

同键同内容返回原结果

同键不同内容返回冲突

统计读取 TimeEntry 的有效记录

AnalysisRun → PlanDraft

输入快照 / 父任务版本 / 分析状态 / 模型与规则版本

草稿可编辑；保存草稿修订与原始输出

WriteReceipt 统一承接原方案 PlanApplyReceipt 的采纳回执

只读统计结果 · 不再建立第二份实际时间账本

task_actual = 任务本身有效记录 + 所有子步骤有效记录

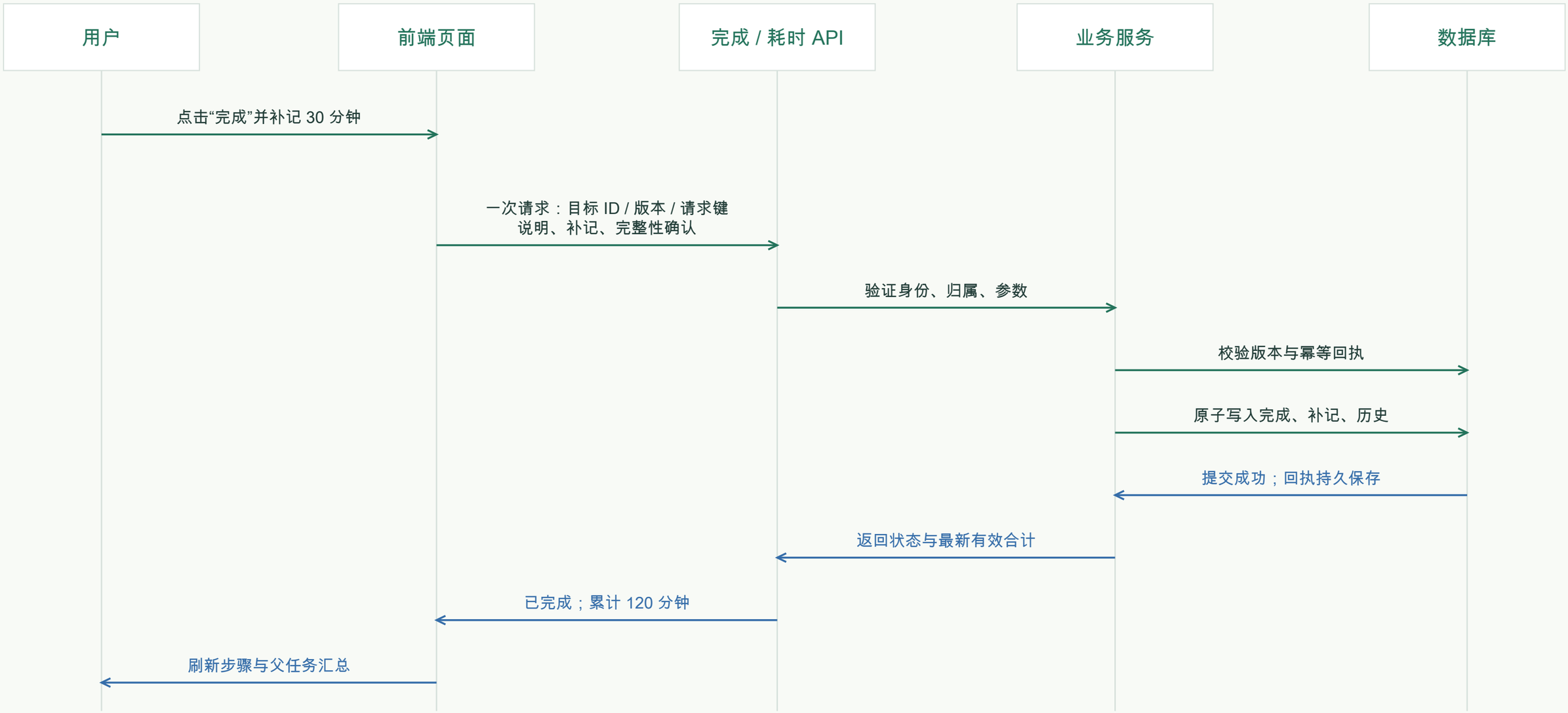
step_actual = 该步骤的有效记录；作废 / 修订历史不计入

无记录为 null；已记录合计与是否完整分别返回

约束：step_id 若非空，必须属于同一个 task_id；从父任务验证 owner；完成、暂停或撤销都不删除已花时间。

前后端交互：一次“完成 + 补记”怎么保存

示例：这个步骤已记 90 分钟，用户发现漏记 30 分钟；提交后显示 120 分钟，不另填一笔 120 分钟。



网络超时 / 重复点击

重试复用同一个请求键与原始内容

后端返回原回执，不重复累计 30 分钟

版本冲突 / 数据库失败

冲突保留输入，刷新后由用户重新确认

事务失败回滚，不能留下半条完成记录

独立记时 / 完成撤销

平时记时不改变任务完成状态

撤销完成保留耗时，重新打开后可继续记

计算与状态规则：把“预计、实际、周期”分清

完成跨越了两天，不等于投入了 48 小时；耗时累计真实投入，周计划衡量每周愿意投入多少。

每周计划投入

父任务：4 小时 / 周
用于后续整体负载与周预算
不与步骤总耗时相加

预计总耗时

步骤预计区间之和：5 ~ 9 小时
只表示预估，未知项单独提示
不能直接当作已花时间

已记录实际耗时

概念 40 + 手算 50 + 代码 90 分钟
父任务累计：180 分钟 = 3 小时
此时父任务不再补记一笔 3 小时

账本归属与后补规则

有子步骤：优先记到步骤；整体整理等工作可记父任务本身。
父任务本身另花 20 分钟，则累计 180 + 20 = 200 分钟。
已记 90 分钟，实际为 120：补记 30 或修正原记录。
若先记在父任务后拆步骤：可迁移归属，不能复制时长。
历史补记允许用于暂停 / 已结束任务，不自动恢复任务状态。

状态与完成记录

必要步骤全部完成 → 提示用户确认父任务；不自动结束。
父任务提前结束 → 明示未完成步骤，不批量假装完成。
父任务暂停 → 子步骤显示随任务暂停，保留原始状态。
撤销完成 → 恢复到完成前的非结束状态，并追加事件。
父任务仍已结束时，先恢复父任务，再恢复步骤执行。

完整性与撤销

没有耗时记录：显示“未记录”，不按 0 分钟处理。
有记录但未核对：显示“已记录合计 · 待确认”。
用户确认完整后才显示最终实际耗时；后续改账需重新确认。

拆分改动与分析

本期重新拆分只支持明确追加，不自动替换已执行步骤。
完成数占比不等于时间完成度，剩余耗时需重新估计。
AI 可给预估与建议；实际投入与完成仍由用户登记确认。

第二部分实施顺序

2A-1 手工步骤与完成标准 → 2A-2 耗时账本、完成登记与汇总 → 2A-3 规则分析与采纳

2B 接入 AI 草稿与后台分析任务；计时器、全局负载与自动排期后续实现。

依据：原始需求分析、第二部分方案 v0.1、当前 Task / TaskEvent 代码及本次完成与耗时讨论。